



Callable & Future



Callable una interfaz funcional similar a **Runnable**, pero con dos superpoderes adicionales: **puede devolver un resultado** y **puede lanzar excepciones comprobadas**. Se usa normalmente junto a **ExecutorService**

Explicación

Mientras que **Runnable** define una tarea *que no devuelve nada*, la interfaz:

```
public interface Callable<V> {  
    V call() throws Exception;  
}
```

define una **unidad de trabajo concurrente** cuyo método **call()**:

- devuelve un valor de tipo **V**,
- y puede lanzar excepciones verificadas (**throws Exception**).

Esto lo vuelve ideal para tareas que **computan un valor**.

Ejemplo simple:

```
Callable<Integer> tarea = () → {  
    return 40 + 2;  
};
```

Por lo general, esta interfaz se usa junto a los ejecutores.

¿Cómo se usa?

Cuando enviamos un **Callable** a un **ExecutorService** con **submit()**, **NO** obtenemos el resultado directamente (porque se está resolviendo de forma asíncrona).

El ejecutor devuelve un objeto `Future<V>` :

- representa el **resultado futuro** de la tarea,
- es como una *promesa* que se cumplirá cuando el hilo termine su trabajo.

El método más importante:

`V get()`

- Es **bloqueante**.
- El hilo llamador se queda esperando hasta que la tarea termine y el resultado esté disponible.

Otros métodos útiles:

- `isDone()` → ¿ha terminado la tarea?
- `cancel(boolean)` → solicita cancelar la tarea si aún no ha empezado o si es cooperativa.
- `isCancelled()` .

Mientras que `Callable` describe *qué* trabajo se debe hacer, **Future** es el objeto que permite obtener el *resultado* de ese trabajo asíncrono, actuando como un marcador de posición para un valor que aún no está disponible.

```
import java.util.concurrent.*;

public class DemoCallableFuture {
    public static void main(String[] args) throws Exception {
        ExecutorService pool = Executors.newSingleThreadExecutor();

        Callable<Integer> tarea = () → {
            System.out.println("Calculando...");
            Thread.sleep(1000);
            return 21 * 2;
        };

        Future<Integer> futuro = pool.submit(tarea);
```

```

        System.out.println("La tarea está en ejecución...");
        System.out.println("¿Ha terminado? " + futuro.isDone());

        Integer resultado = futuro.get(); // BLOQUEA hasta que termine
        System.out.println("Resultado = " + resultado);

        pool.shutdown();
    }
}

```

Ejemplo con lambda

```

ExecutorService pool = Executors.newFixedThreadPool(3);

Callable<String> t1 = () → "Tarea 1";
Callable<String> t2 = () → "Tarea 2";
Callable<String> t3 = () → "Tarea 3";

List<Callable<String>> tareas = List.of(t1, t2, t3);

List<Future<String>> resultados = pool.invokeAll(tareas);

for (Future<String> f : resultados) {
    System.out.println(f.get());
}

pool.shutdown();

```

¿Por qué existe Future?

Porque las tareas enviadas a un `ExecutorService` se ejecutan **asíncronamente**.

El `submit()` devuelve inmediatamente, pero el trabajo aún no ha terminado. Por eso necesitamos un objeto que nos permita:

- consultar el estado,
- esperar el resultado,

- cancelar la tarea.

Ese objeto es `Future`.

Runnable VS Callable

Ambos representan tareas concurrentes, pero **no son equivalentes**.

Característica	Runnable	Callable
¿Devuelve valor?	✗ No	✓ Sí
¿Puede lanzar excepción verificada?	✗ No	✓ Sí
Método	<code>run()</code>	<code>call()</code>
¿Se usa con ejecutores?	Sí (pero sin obtener retorno)	Sí, y devuelve <code>Future</code>